



Módulo 3 – Comunicação Serial e Conv. A/D

ORIENTAÇÕES:

O módulo 3 trata de comunicação serial e conversão de sinais analógicos em digitais. Este módulo vale 25 pontos. Toda semana será selecionado um exercício para receber visto. Serão 4 vistos ao total. Cada visto vale 3 pontos. No final do módulo, um problema será proposto para sumarizar o conteúdo do módulo. O problema vale 13 pontos.

OBJETIVOS DESTA MÓDULO:

- Compreender protocolos de comunicação serial síncrona e assíncrona.
- Diferenciar bit-banging do uso do periférico dedicado.
- Entender o processo de amostragem e conversão de um sinal analógico em digital.
- Configurar as interfaces seriais e o conversor A/D.

Comunicação Serial

Comunicação serial é o processo de envio de dados de maneira sequencial, um bit após o outro. Esse tipo de comunicação permite economizar (preciosos) pinos de entrada e saída ao realizar transferências de grandes quantidades de dados de maneira serial. Ao invés de 16 pinos para enviar uma palavra de 16-bits é necessário apenas 1.

Em sistemas embarcados, dispositivos com funcionalidade específica geralmente implementam algum tipo de comunicação serial para enviar ou receber dados e instruções. Acelerômetros, cartões de memória, GPS, sensores de temperatura, umidade e pressão, todos possuem internamente um microcontrolador que trata o dado bruto e o envia por uma linha serial. Diante disso, é de suma importância compreender as diferentes formas de comunicação para interfacear o microcontrolador com outros dispositivos.

Podemos enquadrar a comunicação serial em duas classes:

- **Síncrona**, na qual é disponibilizado um relógio que indica o instante de validade do bit. Exemplos: SPI, I2C
- **Assíncrona**, quando não há a disponibilidade deste relógio. Exemplos: UART, USB.

Comunicação Serial Síncrona: I²C

O protocolo I²C (*Inter-Integrated Circuit Communication*) permite que vários dispositivos se conectem a um mesmo barramento e se comuniquem entre si. Esse barramento, também conhecido como TWI (*Two Wire Interface*) utiliza apenas dois fios para realizar a comunicação bidirecional entre diversos dispositivos. Na terminologia utilizada, o dispositivo que está gerando uma mensagem é chamado de transmissor, enquanto o dispositivo que está recebendo a mensagem é chamado de receptor. O dispositivo que controla o barramento é chamado de mestre (master), e os dispositivos que respondem ao mestre são chamados de escravos (slave). Os dispositivos mestre e escravo assumem o papel de transmissor ou receptor em diferentes etapas da comunicação. O fio do barramento que carrega dados é chamado de SDA (Serial DAta). O outro fio estabelece o ritmo de comunicação e é chamado de SCL (Serial CLock). Apenas o mestre controla o clock, mesmo quando estiver recebendo uma mensagem.

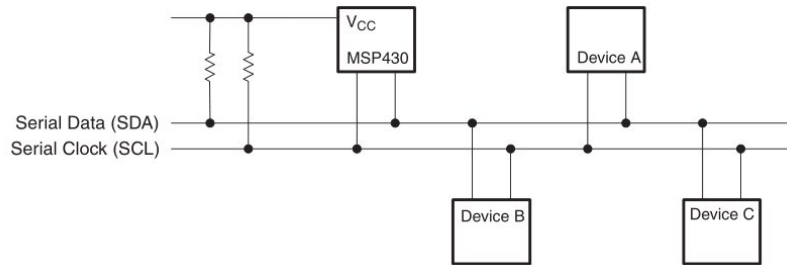


Figura 1 – Barramento I2C

Sinais de controle: Start e Stop

Quando o barramento está ocioso as linhas SDA e SCL ficam em nível lógico HIGH. Isso é garantido através de resistores de pull-up como os da Figura 1. Para iniciar uma transmissão, o mestre força a descida de SDA (mudança de HIGH para LOW) enquanto o clock está em nível alto. Este sinal é a condição de partida (START), e prepara os dispositivos para a comunicação. Para finalizar a transmissão, o mestre libera SDA (mudança de LOW para HIGH) quando o clock está em nível alto. Este sinal é a condição de parada (STOP) liberando assim o barramento para outras comunicações. Se for necessário reestabelecer a comunicação, deve-se gerar novamente a condição de partida. A Figura 2 apresenta as condições de START e STOP.

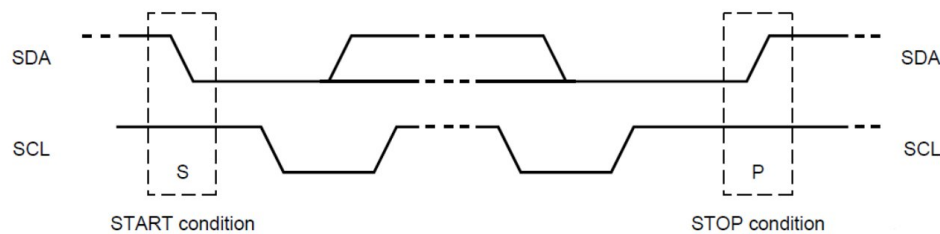


Figura 2 – Comandos de Start e Stop

Transferência de 1 bit

Na transferência de um bit, como mostrado na Figura 3, o dado em SDA deve permanecer estável entre a subida e a descida do clock (SCL). Mudanças em SDA enquanto SCL está em nível alto são entendidas como sinais de controle (como visto nas condições de START e STOP). Durante a transmissão, o valor de SDA é constante enquanto SCL estiver em nível HIGH e assume o valor do bit a ser transmitido.

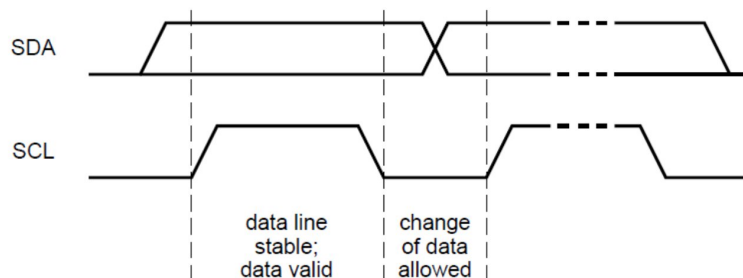


Figura 3 – Transferência de 1 bit



Transferência de uma palavra (8-bits) + ACK/NACK

A comunicação no barramento é feita em bytes (8 bits). A cada byte transmitido, um sinal de *acknowledge* (ACK) deve ser gerado. Portanto, para cada byte transmitido, 9 batidas de clock precisam ser geradas. O número de bytes enviados entre um sinal de START e STOP é ilimitado. A figura 4 mostra a comunicação entre dois dispositivos. O transmissor envia os 8 primeiros bits e libera a linha no nono bit. O receptor então puxa a linha para LOW indicando que entendeu a mensagem, com um ACK. Se a linha ficar em HIGH na nona batida do relógio significa um “*not-acknowledge*” (NACK) e pode ter diversos significados: pode ser que o escravo não entendeu a palavra enviada; ou que não há nenhum dispositivo na linha escutando; ou que o escravo decidiu enviar um NACK de propósito para indicar que não quer ou não pode mais receber bytes.

Na Figura 4, as ações do mestre-transmissor e escravo-receptor são apresentadas separadas, porém ambos compartilham a mesma linha SDA. Isso deixa claro que é o escravo (e não o mestre) que envia o ACK no momento certo. Note que o clock é controlado pelo mestre, inclusive na batida do ACK. Outra curiosidade interessante, é que no momento do ACK, o escravo deixa de ser receptor para transmitir o bit de ACK/NACK e o mestre deixa de ser transmissor para receber o bit de ACK/NACK. Isso reforça a afirmação anterior que diz que o mestre e escravo assumem papéis de transmissor e receptor em diferentes estágios da comunicação.

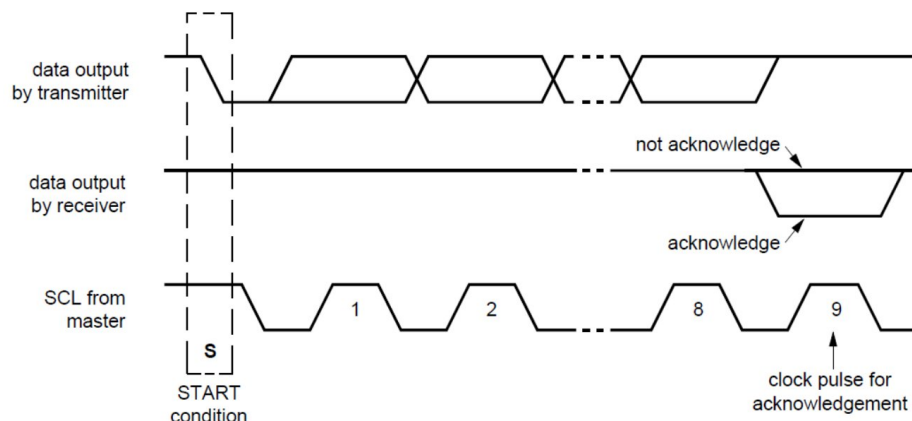


Figura 4 - Transferência de 1 byte

Endereçamento

O primeiro byte enviado logo após a condição de START é especial e é interpretado como endereço do escravo. São enviados 7 bits de endereço e um bit de leitura/escrita que indica se o mestre vai ler ou escrever no escravo, como mostrado na Figura 5. O endereço é enviado na parte mais significativa do primeiro byte e o bit de R/W é enviado por último, ou seja, ele é o LSB.

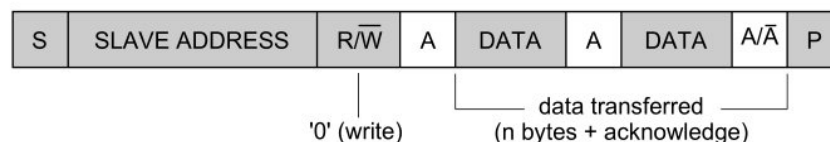


Figura 5 - Endereçamento de um escravo.

Após o envio dessa palavra inicial, as próximas são sempre bytes (8 bits) seguidos do ciclo de ACK/NACK. Por fim, termina-se a comunicação com uma condição de STOP (P).



No MSP430 usamos a interface dedicada USCI, sigla para Universal Serial Communication Interface. Ela é capaz de se comunicar em UART, SPI e I2C. Nesta primeira parte vamos estudar apenas o protocolo I2C. A interface pode ser configurada pelos registros UCBxCTL1 e UCBxCTL0 mostrados na Figura 6:

UCBxCTL0							
7	6	5	4	3	2	1	0
UCA10	UCSLA10	UCMM	Reserved	UCMST	UCMODEx		UCSYNC
rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	r-1

UCBxCTL1							
7	6	5	4	3	2	1	0
UCSSELx		Reserved	UCTR	UCTXNACK	UCTXSTP	UCTXSTT	UCSWRST
rw-0	rw-0	r0	rw-0	rw-0	rw-0	rw-0	rw-1

Can be modified only when UCSWRST = 1.

Figura 6 - Registros de configuração

Perceba que alguns bits (hachurados) só podem ser escritos enquanto a interface estiver resetada, ou seja, **UCSWRST** = 1. Para o modo I2C, devemos fazer **UCMODE** = 3. O bit **UCMST** indica se a interface vai se comportar como mestre ou escravo, ou seja, se vai controlar o clock ou só escutar o clock.

A configuração deve ser feita na seguinte ordem: (1) Resete a interface fazendo o bit **UCSWRST** = 1. (2) Configure os registros de controle CTL0 e CTL1 evitando zerar o bit de reset. (3) Configure os pinos para usar a funcionalidade dedicada (PxSEL). (4) Desative o reset fazendo **UCSWRST** = 0. (5) Opcionalmente, habilite as interrupções que forem necessárias.

Configuração como mestre: Quando a interface for configurada como mestre, devemos escolher o clock nos bits **UCSSEL**. A frequência do clock pode ser dividida pelo valor escrito em **UCBxBRW**. O bit **UCTR** configura a interface como transmissora (1) ou receptora (0). Os bits **UCTXSTT** e **UCTXSTP** são usados para enviar a sinalização de start e stop, respectivamente. O endereço do escravo que o mestre quer se comunicar deve ser escrito em **UCBxI2CSA** (Slave Address). O valor deve estar disponível antes da sinalização de start. Durante a comunicação, os bytes podem ser enviados escrevendo em **UCBxTXBUF** e recebidos em **UCBxRXBUF**. As flags de interrupção podem ser encontradas no registro **UCBxIFG**. A flag **UCTXIFG** sinaliza quando o buffer de transmissão estiver vazio e pode ser escrito. Já a flag **UCRXIFG** sinaliza quando o buffer de recepção estiver cheio e deve ser lido. Recomenda-se uma implementação por polling no mestre.

Configuração como escravo: Não há a necessidade de configurar o clock quando a interface for configurada como escravo. O endereço do escravo deve ser configurado no registro **UCBxI2COA** (I2C Own Address). Recomenda-se uma implementação a base de interrupções para o escravo. Os eventos de TX, RX, NACK, STT e STP podem ser decodificados a partir do vetor de interrupções (IV) associado à interface.

O MSP430F5529 possui duas instâncias da USCI chamadas de B0 e B1. A interface B0 está mapeada nos pinos P3.0 (SDA) e P3.1 (SCL). Já a interface B1 está nos pinos P4.1 (SDA) e P4.2 (SCL). Você deve habilitar os resistores de pull-up internos (e configurar **DIR=0**) para garantir o bom funcionamento da interface ou usar resistores externos. Recomendamos não ultrapassar 100kHz para o clock. Os resistores internos do MSP430 são elevados e limitam a velocidade de operação da linha.



Exercício 1: Teste da interface I²C.

Neste módulo, iremos usar o LCD de 2x16 caracteres, como mostrado na Figura 7. Neste primeiro exercício iremos apenas ligar e desligar a retroiluminação do display num ritmo de 1Hz. Configure a interface USCI-B0 como mestre **como descrito na página 4**. Configure os pinos P3.0 e P3.1 para usar a função dedicada dessa interface (PxSEL = 1). *Não se esqueça de habilitar os resistores de pull-up ou coloque resistores externos.*

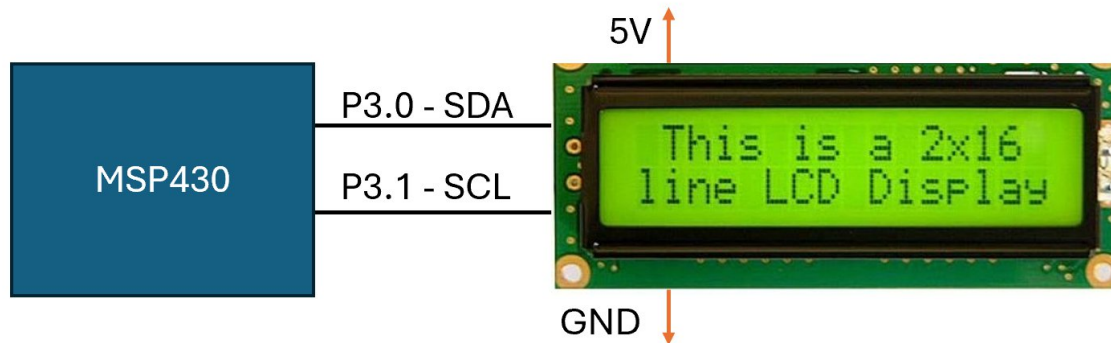


Figura 7 – MSP430 conectado ao LCD.

Em seguida, escreva a função **uint8_t i2cSend(uint8_t addr, uint8_t data)** para fazer o mestre enviar um byte para o escravo. O primeiro byte é o endereço do escravo e o segundo é o byte que deve ser enviado. A função deve retornar 0 se houve ACK e 1 se houve NACK. Implemente essa função seguindo as etapas abaixo:

1. Escreva o endereço do escravo em I2CSA
2. Faça TR = 1 e STT = 1 para requisitar um start como transmissor
3. Escreva o byte **data** em TXBUF quando o buffer de transmissão ficar vazio (TXIFG = 1) para destravar o ciclo de ACK
4. Espere o ciclo de acknowledge observando quando STT volta para zero.
5. Se o escravo respondeu (NACKIFG = 0),
 - a. Espere o byte **data** ser transmitido (TXIFG=1)
6. Por fim, envie um stop fazendo STP = 1
7. Aguarde STP voltar para zero para garantir que o stop foi enviado
8. Retorne a variável que sinaliza se houve um ACK ou NACK (NACKIFG)

Atenção: existem duas versões do chip PCF8574 com endereços distintos. Faça uma tentativa para ver qual o endereço de seu chip.

- PCF8574A à 0x3F e
- PCF8574T à 0x27

Faça a retroiluminação do LCD piscar em aproximadamente 1 Hz. Mande o byte 0x00 para desligar a retroiluminação do LCD e 0x08 para acendê-la.

Sugestão extra: Generalizar a função i2cSend(..) para enviar nBytes.



Exercício 2: Scanner de endereços I²C

Aproveite a configuração do exercício anterior para implementar um scanner de endereços do barramento I²C `i2cScan(uint8_t * addrs)`. O mestre deve varrer todos os endereços disponíveis no barramento e escrever num vetor todos os endereços dos dispositivos que responderam com ACK. DICA: Se você requisitar start e stop ao mesmo tempo, o byte de dados não é enviado. É recomendado o uso da técnica de amostragem (polling) das flags da interface USCI ao invés de interrupções.

Liquid Crystal Display (LCD)

É comum utilizar displays de LCD, como o da Figura 8, em aplicações com microcontroladores. A maioria dos displays do mercado utiliza o controlador HD 44780.



Figura 8 - LCD de 2 linhas e 16 colunas de caracteres.

Esse display tem 16 pinos de interface – 2 pinos de alimentação (Vss, Vdd), 1 para o ajuste de contraste (Vo), 2 para o backlight (A, K), 8 pinos de dados D[7:0] (1 byte) e 3 pinos de controle (RS, R/W e EN). O controlador do LCD aguarda um pulso em EN (Enable) para realizar a leitura/escrita dos dados nos pinos D[7:0].

A Figura 9 apresenta a temporização para acessar o LCD. É importante seguir esse diagrama de tempo já que o MSP430 é capaz de escrever nas portas do LCD mais rápido do que o próprio LCD consegue ler. Faz-se necessário o uso de pequenos atrasos para garantir a temporização especificada.

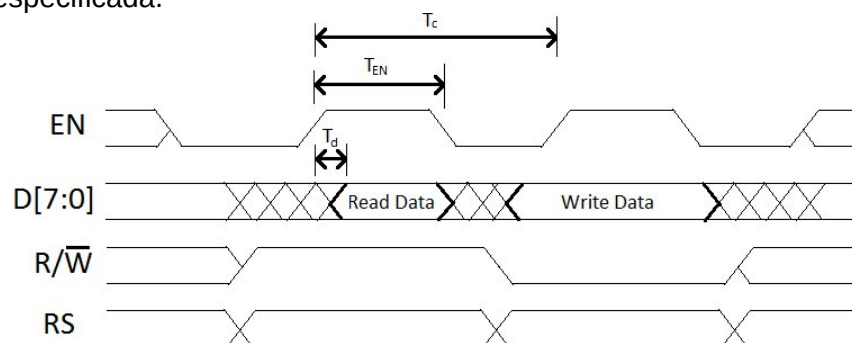


Figura 9 – Temporização de escrita/leitura no LCD

A duração do pulso de enable não deve ser menor que $T_{EN} = 300ns$ e deve ser espaçado de $T_C \geq 500ns$. Em modo de leitura, o dado fica disponível em D[7:0] $T_d = 90ns$ após o pulso do



signal enable. Em modo de escrita, o dado deve estar presente em D[7:0] antes da subida do pulso de enable.

Tirando os pinos de alimentação, o protocolo completo exige o uso de 11 pinos, entretanto, a quantidade de pinos de um microcontrolador é um recurso limitado e geralmente escasso. Alguns microcontroladores do mercado possuem apenas 6 pinos utilizáveis. Com tão poucos pinos, é impossível implementar o protocolo LCD em modo paralelo. Uma alternativa muito comum é usar a configuração do LCD para barramento de dados de 4 bits. Neste caso, a transação de cada byte é feita com 2 acessos de 4 bits, o primeiro para os MSBs e o segundo para os LSBs. A Figura 10 mostra o caso de uma escrita no LCD.

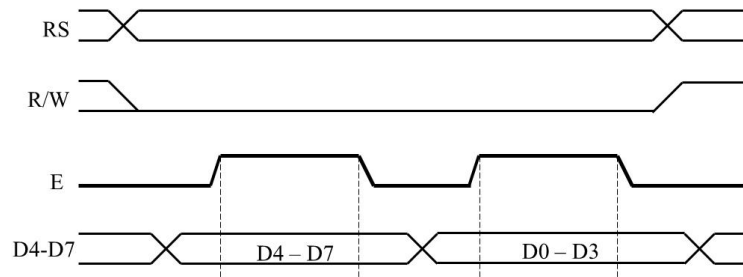


Figura 10 – Escrita de 1 byte no LCD em modo 4 bits.

No modo de operação em 4 bits, são necessários apenas 7 pinos, que podem ser reduzidos para 6 se as operações forem apenas de escrita. Se a quantidade de pinos ainda for um fator limitante do projeto, pode-se usar um conversor de serial para paralelo através de um registrador de deslocamento com portas bidirecionais.

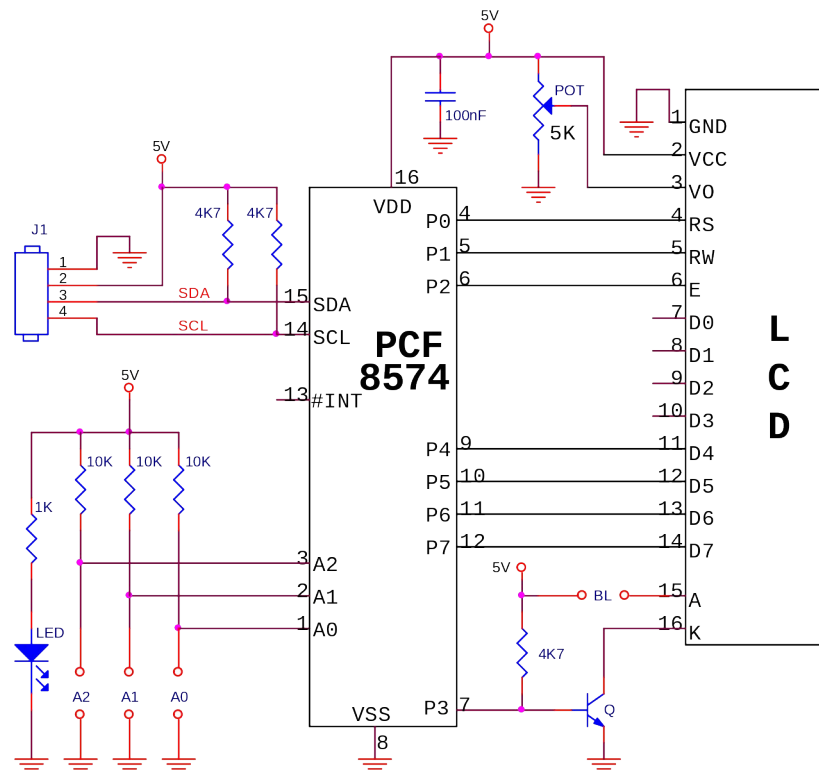




Figura 11 – Esquemático do conversor serial paralelo e conexão com o LCD

O circuito integrado PCF8574 faz exatamente isso. Ele possui uma interface serial síncrona I2C que escreve num registrador de deslocamento transformando dados enviados de maneira serial num barramento paralelo de 8 bits. Desses 8 bits, 3 são usados para controle (RS , $R/\bar{W}$ e EN), 4 bits para o barramento de dados reduzido (modo de 4 bits) e 1 bit para ligar e desligar a retroiluminação (backlight) do LCD, como mostrado na Figura 11, no pino 16 do LCD.

Inicialização do LCD

A escrita de um caractere é feita com $RS = 1$. Uma escrita com $RS = 0$ permite enviar instruções. A leitura com $RS = 0$ permite ler a posição atual do cursor e o bit indicativo de ocupado (BUSY FLAG) que é colocado na posição mais significativa (BIT7). Apenas as instruções essenciais do LCD estão listadas na tabela 1. A lista completa consta no datasheet do controlador Hitachi.

Tabela 1 – Comandos do LCD ($RS=0$, $R/\bar{W}=0$).

Comando	Valor da instrução (binário)								Explicação
Limpar display	0	0	0	0	0	0	0	1	Apaga todos os caracteres e retorna o cursor para o início (leva 1,53ms para executar)
Retorna o cursor para o início	0	0	0	0	0	0	1	-	Retorna o cursor para o início (leva 1,53ms para executar)
Configura o display	0	0	0	0	1	D	C	B	D=0 Desliga o display, D=1 liga o display, C=0 Desliga o cursor, C=1 liga o cursor, B=0 Cursor estático, B=1 Cursor piscando
Configura a interface	0	0	1	M	L	F	-	-	M=0 Modo 4 bits, M=1 Modo 8 bits L=0 Display de 1 linha, L=1 Usa 2 linhas F=0 Fonte 5x8, F=1 Fonte 5x11
Seta o endereço da CGRAM	0	1	A_5	A_4	A_3	A_2	A_1	A_0	Configura que a próxima escrita de dados com $RS = 1$ será na CGRAM a partir do endereço de 6 bits $A[5:0]$.
Seta o endereço da DDRAM	1	A_6	A_5	A_4	A_3	A_2	A_1	A_0	Configura que a próxima escrita de dados com $RS = 1$ será na DDRAM a partir do endereço de 7 bits $A[6:0]$.

Para inicializar o LCD, precisamos colocá-lo em modo 4-bits. Entretanto, como o estado do LCD é desconhecido no início do programa, precisamos fazer com que o LCD retorne ao modo 8 bits para garantirmos que ele comece de um estado conhecido. Para fazer isso, precisamos enviar a instrução 0011.xxxx (veja a Tabela 1). No início do programa do MSP430, o controlador do LCD pode estar num dos estados abaixo:

1. O LCD está em modo 8 bits
2. O LCD está em modo 4 bits esperando o MSNibble
3. O LCD está em modo 4 bits esperando os LSNibble



O primeiro caso é o melhor, pois já estamos no estado que queremos, basta enviar a instrução para configurar o modo 4-bits. Se estivermos no segundo caso, não precisamos voltar para o modo 8-bits, basta continuar configurar o display e o cursor. No terceiro caso, não sabemos qual foi o MSNibble enviado, então enviamos um nibble qualquer (dummy) para completar a última palavra e em seguida enviamos mais dois nibbles para enviar uma instrução de configuração.

Como não sabemos em qual estado o LCD está, precisamos de alguma estratégia para garantir que o LCD entre num estado conhecido. **Vamos enviar o nibble 0x3 três vezes** para garantir que o estado inicial seja o modo 8 bits. O leitor é convidado a testar mentalmente essa solução nos 3 casos. Note que ela funciona para todos os casos.

Com o LCD em modo 8-bits, prosseguimos enviando a instrução para configurar a interface em modo 4-bits e as instruções de configuração do display e cursor. Por fim, limpamos o display e estamos prontos para escrever caracteres. Note que a instrução que limpa o display leva 1,53ms para executar. **É recomendado o esperar esse tempo ao final da inicialização antes de escrever caracteres no display.**

Exercício 3: Escreva uma função **LcdWriteNibble(uint8_t nibble, uint8_t isChar)** para escrever um nibble no LCD em modo 4 bits seguindo o diagrama ilustrado na Figura 10. Faça uso da função **i2cSend(...)** 3 vezes da seguinte forma:

1. Prepare RS (isChar) e D[7:4] com EN=0 e R/W = 0 (write)
2. Em seguida mantenha R/W, RS e D[7:4] constantes e faça EN=1.
3. Por fim, faça EN voltar a 0 mantendo os outros campos constantes.

A variável **isChar** deve representar o valor de RS, ou seja, se for 1 o nibble representa parte de um caractere, se for 0 o nibble é parte de uma instrução.

Exercício 4: Escreva a função **LcdWriteByte(uint8_t byte, uint8_t isChar)**, chamando **LcdWriteNibble** duas vezes. O primeiro nibble enviado corresponde aos 4 bits mais significativos do byte conforme descrito na Figura 10.

Exercício 5: Escreva a função **LcdInit()** que inicializa o LCD seguindo o que foi descrito na seção “Inicialização do LCD”. Note que você irá utilizar a função **LcdWriteNibble** três vezes para garantir que o LCD parte de um estado conhecido, ou seja, do modo 8 bits. Em seguida, coloca-se o LCD em modo 4 bits chamando a função **LcdWriteNibble** uma quarta vez. A partir daí devemos usar apenas a função **LcdWriteByte** para escrever as próximas instruções. Lembre-se que toda instrução é escrita com RS = 0, ou seja, isChar = 0.

Exercício 6: Escreva a função **LcdWrite(char *str)** que escreve uma string no LCD e gerencia a quebra de linha. Lembre-se que uma string é representada por uma sequência de caracteres que termina no byte 0x00.

Exercício 7: Repita a lógica dos exercícios 3 e 4 para fazer a leitura de um nibble e depois de um byte numa função **LcdReadByte(...)**. O momento ideal de leitura é no meio do pulso do EN.



Exercício 8: Use a rotina do exercício 7 para escrever a função `uint8_t lcdBusy()` e verificar se o LCD está ocupado. A função não recebe nenhum parâmetro de entrada e deve retornar o valor do bit busy.

Conversor A/D e DMA

Conversores A/D são dispositivos de interface entre o mundo analógico e o mundo digital. Eles permitem que um microcontrolador colete dados de transdutores que convertem grandezas físicas (como som, luz, temperatura, pH) no formato de tensão.

Neste roteiro, iremos estudar o princípio da conversão A/D. Para isso, utilizaremos divisores resistivos (na forma de potenciômetros) na entrada do conversor. Dessa forma, iremos realizar leituras de tensões proporcionais à posição do potenciômetro.

Um dispositivo simples que aplica esse princípio é o manche de um joystick. Esse dispositivo é tipicamente composto por dois potenciômetros, um para o eixo x e outro para o eixo y, como mostrado na Figura 12. Os dois terminais de leitura da posição do eixo x e y são na verdade valores de tensão entre uma referência positiva (ex: 3,3V) e uma referência negativa (ex: 0V).

Importante: ao conectar o joystick à LaunchPad, o pino rotulado de +5V deve ser ligado ao pino 3,3V. NÃO O LIGUE AOS 5V. Um sinal acima de 3,3V pode queimar o pino do MSP430.

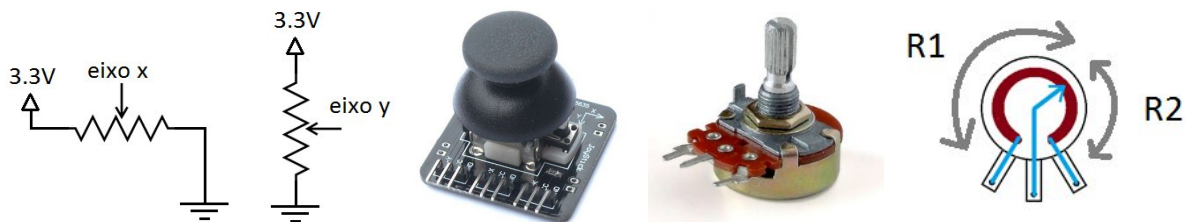


Figura 12 – Joystick com dois potenciômetros e visualização interna de um pot.

O MSP430 possui um conversor A/D integrado de até 12 bits que funciona por aproximações sucessivas. Num primeiro momento, o sinal de entrada é amostrado e a tensão da entrada carrega um capacitor que guarda essa tensão. Numa segunda etapa, o conversor testa bit a bit, começando do MSB até o LSB para descobrir qual é o valor binário que melhor representa aquele nível de tensão. A Figura 13 mostra as etapas de conversão de um caso simplificado de apenas 3 bits.

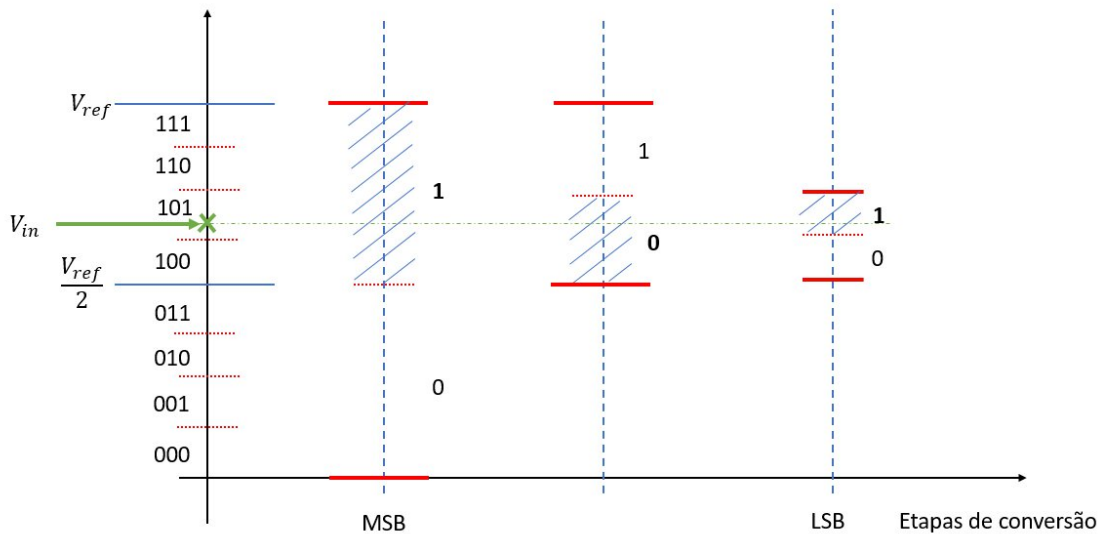


Figura 13 – Etapas de conversão de um conversor por aproximações sucessivas

No exemplo da Figura 13, a tensão de entrada V_{in} está acima da metade da referência. Assim, na primeira etapa de conversão, o conversor A/D atribui o valor “1” ao MSB. Em seguida, o conversor divide a escala da etapa anterior (região hachurada) em duas e verifica se a entrada se encontra na parte superior ou inferior. No exemplo a tensão se encontra na parte inferior da segunda etapa e então o segundo bit é “0”. Por fim, repete-se esse procedimento até que todos os bits sejam definidos. No exemplo, a palavra binária que representa a tensão V_{in} é **101**.

O MSP430 disponibiliza um conversor A/D de 12 bits, compartilhado entre 16 entradas, como mostrado na Figura 14. Somente uma entrada analógica por ser convertida a cada vez.

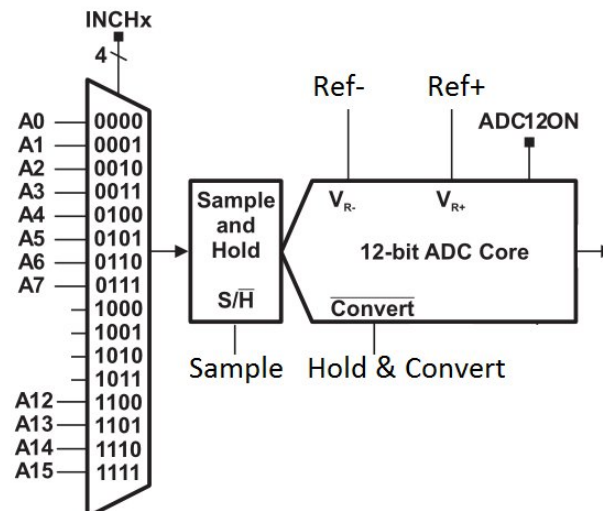


Figura 14 – Conversor AD do MSP430.

Para que a conversão seja feita em etapas sucessivas, é necessário que a entrada V_{in} esteja estável durante todo o processo de conversão. Isso é garantido através de um capacitor na entrada do conversor. Devido à natureza desse elemento, o capacitor leva um certo tempo para acumular carga, então é necessária uma etapa de amostragem do sinal antes de iniciar a conversão. Esse processo é conhecido como *sample and hold*. A Figura 15 modela a entrada do conversor A/D. O resistor R_i representa a resistência de entrada do multiplexador e o capacitor C_i representa o bloco “Sample and Hold”.

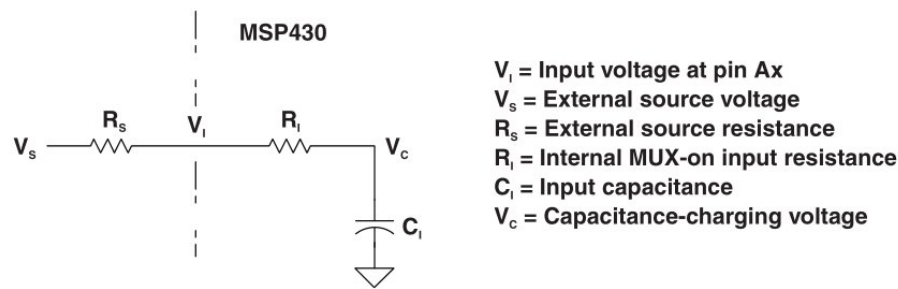


Figura 15 – Amostragem da entrada

A resistência de saída do dispositivo externo (R_s) conectado à porta do conversor AD influencia o tempo de amostragem, ou seja, o tempo que leva para o capacitor da entrada (C_i) carregar. Inicie seu projeto calculando o tempo de carga do capacitor de entrada do conversor A/D seguindo a seguinte expressão:

$$t_{cap} = (R_s + R_i) \cdot C_i \cdot \ln(2^{N+1}) + 800ns$$

Considere a resistência interna de $1,8k\Omega$ (resistência do mux de entrada do conversor) e o capacitor interno de $25pF$. A resistência R_s deve ser o valor do potenciômetro no pior caso de medida, ou seja, em 50%. Para pequenas variações (quando alteramos a posição do potenciômetro) o MSP “enxerga” dois caminhos para a referência e no pior caso a resistência vista pelo MSP é dada por:

$$R_s = \frac{R_{pot}}{2} \parallel \frac{R_{pot}}{2} = \frac{R_{pot}}{4}$$

Exercício 9:

O ADC12 possui um timer interno que pode ser usado para garantir o tempo de carregamento do capacitor t_{cap} . Se o potenciômetro tem $10k\Omega$ de resistência total, qual é o valor de t_{cap} ? Consequentemente, se $SHP=1$, qual deve ser o valor de SHT se escolhermos o clock $SMCLK @1MHz$?

Exercício 10:

Escreva a função `uint16_t adcRead(uint8_t pin)` que faz uma leitura pontual de qualquer pino da porta P6. Considere no pior caso, sensores com $50k\Omega$ de resistência. Use 12 bits de resolução. Use o modo *single-channel, single-conversion*.



Exercício 11:

Configure o módulo do conversor A/D em modo *repeated single channel*, para amostrar a tensão de entrada que representa apenas um eixo da posição do joystick. Use apenas 8 bits de resolução. Configure um timer para realizar essa amostragem a cada 100ms. Você pode plotar o resultado da leitura no próprio CCS colocando o vetor na aba de variáveis ou de expressões (Watch). Em seguida, clique no vetor com o botão direito e selecione a opção “Graph Selected Value”, como mostrado na Figura 16.

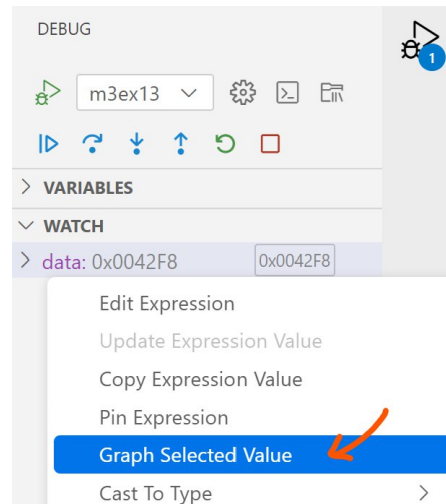


Figura 16 – Plotar um gráfico a partir de um vetor no CCS

Exercício 12:

Repita o exercício anterior sem usar o timer interno do ADC (SHP = 0). Use o canal 1 do timer A0 e o escolha como fonte do gatilho, ou seja, faça SHS = 1. Não há mais necessidade de configurar o SHT. No timer, configure o canal 1 para gerar um sinal PWM e ajuste o valor de CCR1 para se adequar ao tempo de carregamento do capacitor projetado no exercício anterior. Se desejar precisão, use a frequência do SMCLK dividida por 2.

Exercício 13:

Incremente o exercício anterior para fazer a leitura dos dois eixos do joystick. Use o modo sequencial repetitivo e configure o controlador do buffer de memória para ler os dois canais. No último canal, ative o bit de final de sequência (EOS). *Evite usar o bit MSC no modo sequencial repetitivo.*

Exercício 14:

Neste programa, iremos ler simultaneamente (ou quase) o valor da posição de dois eixos do potenciômetro. Use a informação da posição dos dois eixos para controlar a luminosidade dos dois LEDs da LaunchPad. Se a posição do joystick estiver no centro, os dois LEDs devem estar em 50% da luminosidade.

Use o eixo X para controlar a luminosidade do LED vermelho e o eixo Y para controlar a luminosidade do LED verde. Trabalhe com um sinal PWM a 100Hz com 256 passos (de 0 a 255). Ajuste a resolução do conversor A/D para 8-bits de forma a ficar na mesma resolução da PWM. Amostre a posição do potenciômetro a cada 20ms (50Hz). A figura 17 apresenta um diagrama do sistema proposto.

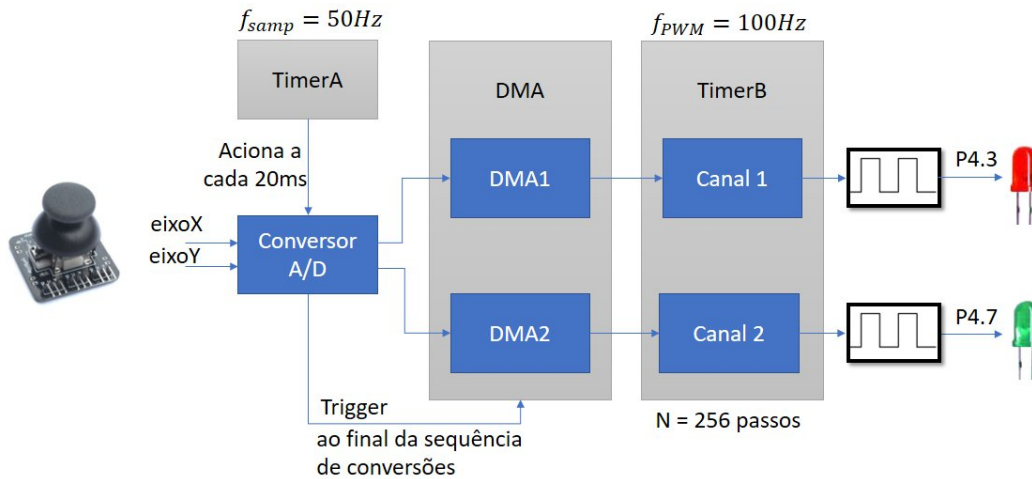


Figura 17 – Controle do brilho dos leds usando um joystick

Neste exercício, utilizaremos a saída dos canais 1 e 2 do timer B para controlar a luminosidade de ambos os LEDs. Para isso, use a porta P4 que permite remapear a funcionalidade dedicada de seus pinos como no exemplo abaixo.

```
// Configure o mapeamento dos pinos da porta P4
PMAPKEYID = 0x02D52; // Escreve a senha para liberar a config do PMAP
P4MAP3 = PM_TB0CCR1A; // Conecta P4.3 na saída do canal 1 do timer B
P4MAP7 = PM_TB0CCR2A; // Conecta P4.7 na saída do canal 2 do timer B
```

Copiar regiões de memória, seja entre periféricos ou entre posições de memória, é uma tarefa trivial e bastante comum em programas de sistemas embarcados. O DMA (Direct Memory Access) é um periférico dedicado para realizar transferência de dados e permite que cópias sejam realizadas sem intervenção do processador. *Neste exercício, utilize o DMA para mover o resultado da conversão para os canais de saída do timer B e consequentemente ajustar a largura do pulso de saída automaticamente (isso irá modificar a luminosidade do LED).*

Se não souber usar o DMA ainda, use a interrupção do ADC para mover os resultados de conversão para o CCR1 e CCR2 do timerB.



Comunicação Serial Assíncrona

A comunicação serial assíncrona é provavelmente a forma serial mais simples de todas. Sua popularidade vem justamente da sua simplicidade e facilidade de implementar usando lógica digital ou microcontroladores.

Para cada linha serial, a comunicação está ociosa quando em nível alto. Para iniciar uma transmissão, é necessário puxar a linha para nível baixo e transmitir bit a bit de maneira sequencial, começando pelo menos significativo. A taxa de transferência é um acordo prévio, ou seja, ambas as partes devem estar configuradas para se comunicar na mesma velocidade.

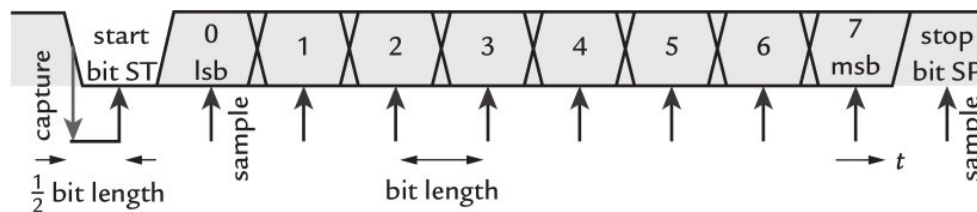


Figura 18 – Comunicação serial assíncrona

Um dispositivo UART (Universal Asynchronous Receiver/Transmitter) não permite comunicação bidirecional através de um único fio. São necessários pelo menos dois fios para uma comunicação full-duplex. Uma particularidade desta forma de comunicação é a ausência de sincronia entre o transmissor e receptor. Isso pode levar a erros de comunicação caso haja diferença significativa entre os clocks internos de cada dispositivo e limita a taxa de comunicação possível entre dispositivos. O máximo recomendado é de aproximadamente 100kbps, entretanto, o valor mais comum é de 9600bps.

O MSP430 possui um módulo de comunicação serial dedicado. Isso torna o processo de comunicação tão simples quanto escrever num registro e esperar que o byte seja enviado pela linha serial. Isso nem sempre acontece com microcontroladores mais simples. Nesses casos o protocolo deve ser implementado manualmente através de leituras e escritas dos pinos. Este tipo de implementação é chamado de *bit-banging* e exige cuidado extra com a temporização para não perder sincronia.

Exercício 15:

A interface serial USCI-A0 está roteada nos pinos P3.3 (TX) e P3.4 (RX). Configure esses pinos para sua funcionalidade dedicada. Conecte os dois como na Figura 19 para testar a interface de comunicação serial. Note que a USCI transmite e recebe o dado transmitido. Para este exercício, e para os próximos, iremos trabalhar à uma taxa (baudrate) de 1200 bps. Teste a interface enviando um byte qualquer e verifique que ele é recebido de volta. Verifique, através de breakpoints nas interrupções, ou por amostragem da flag de recepção (RXIFG), se o valor foi recebido corretamente no buffer de recepção.



Figura 19 – USCI transmitindo pelo pino P3.3 e recebendo o dado transmitido pelo pino P3.4.



Exercício 16:

Ainda com a mesma configuração do exercício anterior, envie e receba o seu nome pela interface serial, caractere por caractere. Use dois vetores:

- `char nome_tx[] = "Joaquim José"` à com o nome a ser transmitido e
- `char nome_rx[] = "xxxxxxxxxxx"` à espaço para receber o nome

Lembre-se de que as strings terminam com zero ('\0'). Use este fato para finalizar tanto a transmissão quanto a recepção.

Exercício 17:

Utilize a interface serial para controlar os LEDs de outro microcontrolador. Ao pressionar o botão S1 o LED vermelho do outro uC deve alternar, já o botão S2 controla o LED verde. A ligação entre dois microcontroladores deve ser feita conforme a Figura 20.



Figura 20 – Conexão para comunicação entre dois MSPs.

O receptor responde aos seguintes códigos:

- `0x01` – Liga o LED **vermelho**
- `0x0A` – **Apaga** o LED **vermelho**
- `0x10` – Liga o LED **verde**
- `0xA0` – **Apaga** o LED **verde**

O transmissor deve enviar apenas um código a cada pressionar de botão:

- S1 – Envia **0x01** e **0x0A** alternadamente
- S2 – Envia **0x10** e **0xA0** alternadamente

Exemplo: Ao pressionar uma vez o botão S1 o transmissor deve enviar a palavra **0x01**. Ao pressionar uma segunda vez, o transmissor envia **0x0A**.

Exercício 18:

Iremos repetir o exercício anterior usando a técnica de *bit-banging*. Este exercício é importante pois nem todos os microcontroladores disponíveis no mercado possuem interfaces de comunicação serial dedicada. Você pode usar os mesmos pinos do exercício anterior, porém em modo GPIO. Mantenha a mesma taxa de transferência que os exercícios anteriores. A recepção dos dados deve ser feita no centro da janela (temporal) de cada bit, conforme a Figura 18, pois assim se evita erros de recepção caso os dispositivos estejam ligeiramente fora de sincronia.

Exercício 19:

A LaunchPad oferece uma conexão UART com o PC que pode ser muito útil para auxiliar na depuração do código. A Figura 21 apresenta um esquema desta conexão serial. Veja que a placa programadora "eZ-FET" funciona como um conversor USB/Serial pelos pinos P4.4 e P4.5 que estão conectados na instância USCI_A1. Esta é uma conexão interna à LaunchPad. Você não precisa ligar cabo algum, apenas use a USCI_A1. No PC, deve ser usado um emulador de terminal serial. Sugerimos o uso do terminal do próprio CCS.

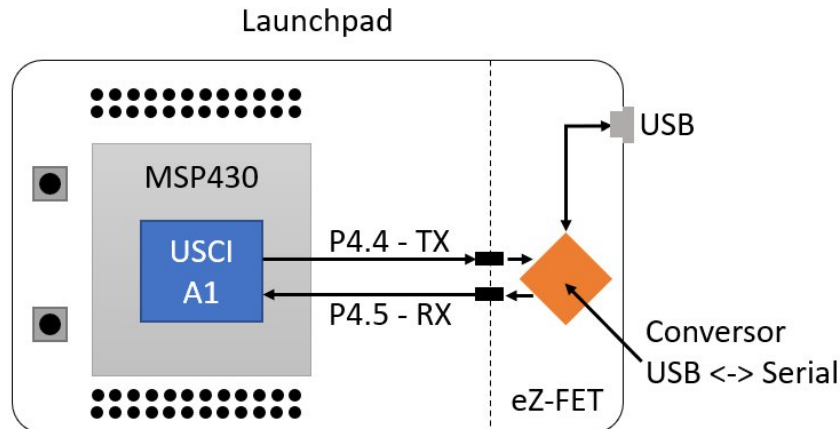


Figura 21 – Conexão serial entre o PC e o MSP.

Quando você conecta o MSP no computador, o Sistema Operacional detecta dois dispositivos UART. Um para depuração e outro para uso geral chamado “MSP Application UART1”. No Windows, você pode descobrir o número da porta (COMx) no gerenciador de dispositivos como ilustrado na Figura 22. No Linux, o dispositivo é criado no arquivo `/dev/ttyACMx`. No Mac é o dispositivo `/dev/tty.usbmodem.xxxx` (escolher o dispositivo de maior número).

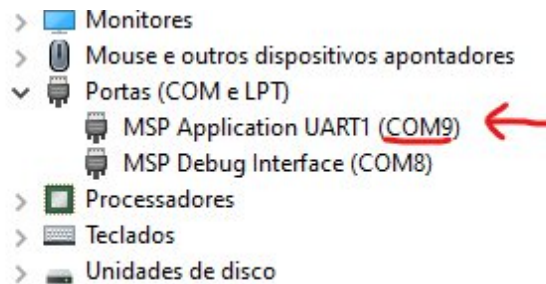


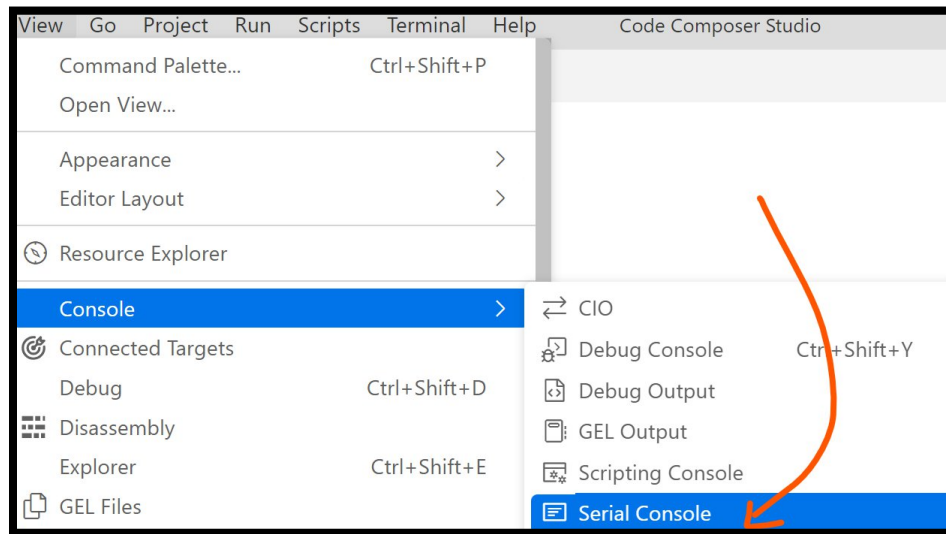
Figura 22 – Seleção da porta (COM) a ser usada no PC.

Para criar um terminal no Code Composer, clique em View à Console à Serial Console. Encontre a janela do terminal “Serial Console” e abra uma nova conexão. Configure os detalhes da conexão da mesma forma que você fez no MSP430. Veja a Figura 23.

Neste exercício vamos criar uma função void **uartPrint**(char * str) que envia uma string pela interface serial USCI_A1 e exibe a string no computador pelo terminal serial do CCS. Vamos gerenciar a recepção por interrupções. Grave o que for recebido num buffer circular de 32 caracteres na ISR da USCI_A1.



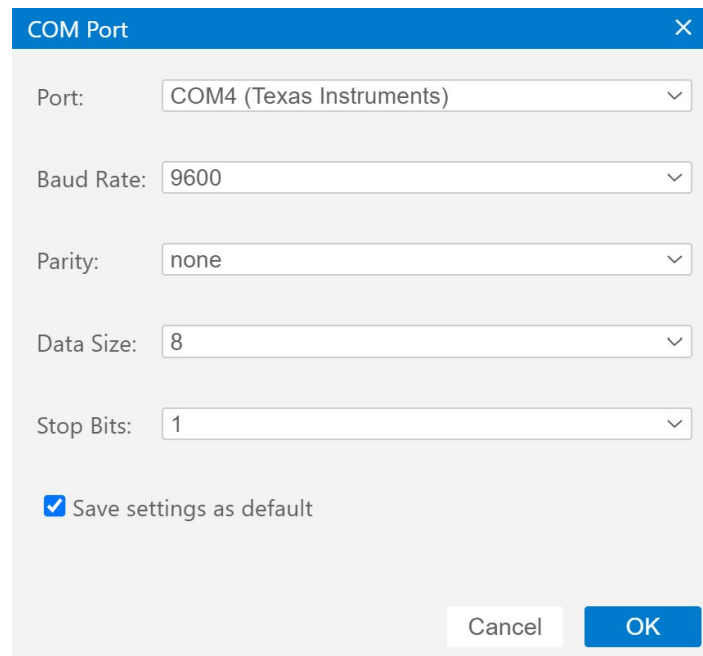
Figura 23: Criação de uma conexão serial com o MSP no CCS



Passo 1 – Abrir a vista do terminal serial



Passo 2 – Abrir uma nova conexão



Passo 3 – Configurar a conexão



Problema 3 – Conversor A/D, LCD e UART

OBJETIVO:

Fazer um osciloscópio digital usando as entradas do A/D do MSP430.

PEDIDO:

Osciloscópios são aparelhos muito úteis para visualizar formas de onda, otimizar, depurar ou simplesmente estudar circuitos. Neste exercício iremos projetar um osciloscópio digital de até 8 entradas usando o MSP430. O usuário poderá selecionar quantos canais quer usar através de uma interface UART pelo PC. Os requisitos de projeto são:

- 1) A resolução deve ser fixa em 12 bits.
- 2) O valor das leituras será mostrado em hexadecimal usando 3 algarismos.
- 3) O MSP pode receber a qualquer momento, um dígito N de 1 a 8 que servirá para (re)configurar o número de canais.
- 4) A UART deve operar em 19200 bps.
- 5) O formato de impressão deve ser c: xxx, onde “c” é o número do canal e “xxx” é o valor lido em hexadecimal. Exemplo com 3 canais:
0: 800
1: FFF
2: 4A3
- 6) No LCD será mostrado apenas o canal A0. O terminal UART mostrará os N canais
- 7) A taxa de amostragem depende do número de canais. Ela deve ser configurada para 200/N amostras por segundo.
- 8) Considere uma resistência externa de 10kΩ.

DICAS:

- a) As sequências de caracteres abaixo podem ser usadas para limpar a tela ou fazer o cursor retornar para o início da linha.

```
#define TERM_CLR "\033[2J"  
#define TERM_HOME "\033[0;0H"
```

- b) Para escrever uma nova linha a partir do início deve-se enviar os caracteres “\n” (nova linha) e “\r” (retorna o cursor para o início da linha).
- c) Você pode usar a função `sprintf(str_out, "%03x", value)` da biblioteca “string.h” para gerar uma string (*str_out*) correspondente a um número (*value*) em hexadecimal com 3 dígitos fixos com preenchimento de zeros à esquerda.
- d) Implemente a transmissão UART por pooling (na main)
- e) Implemente a recepção UART por interrupção
- f) Use um timer para gerenciar o gatilho de amostragem do ADC
- g) Se for usar o joystick para testar, **não conecte a alimentação em 5V**. Lembre-se que o MSP430 opera em 3V3.